

Requerimientos No Funcionales

E-Commerce Zero · Trabajo Práctico Integrador N°1 · Ingeniería del Software II · 2026

Los requerimientos no funcionales describen las cualidades y restricciones del sistema: seguridad, usabilidad, rendimiento, confiabilidad, mantenibilidad, portabilidad y restricciones del negocio. Surgen del enunciado del trabajo práctico y de las decisiones del equipo. Para cada uno se indica cómo se cumple en el sistema desarrollado.

1. Seguridad

ID	Requerimiento	Cómo se cumple
RNF01	Las contraseñas de los usuarios se almacenan encriptadas; nunca se guardan en texto plano.	Encriptación con BCrypt (<i>BCryptPasswordEncoder</i> en <i>SecurityConfig</i>).
RNF02	El ingreso al sistema se realiza con usuario (correo personal) y contraseña. La contraseña debe tener entre 8 y 64 caracteres y confirmarse al registrarse.	Login con Spring Security (formulario <i>/login</i>). Validación en <i>UsuarioService</i> .
RNF03	Una cuenta de cliente nueva queda inactiva hasta que se ingresa el código de activación de 6 dígitos enviado por correo. Las cuentas inactivas o dadas de baja no pueden iniciar sesión.	Atributo <i>codigoActivacion</i> de <i>Usuario</i> ; excepciones <i>ActivacionPendienteException</i> y <i>CuentaEliminadaException</i> .
RNF04	El acceso a cada sección se limita según el rol: JEFE accede a todo el panel; ADMINISTRATIVO a todo el panel salvo usuarios y configuración de empresa/correo; CLIENTE a su perfil, carrito y compras. Un acceso sin permiso muestra un error 403.	Reglas por ruta en <i>SecurityConfig</i> (<i>/admin/**</i> , <i>/cliente/**</i>) y menús según el rol.
RNF05	Un cliente solo puede ver y operar sobre sus propios datos (carrito, compras y perfil), aunque modifique los identificadores en la URL.	Verificación del cliente dueño en <i>OrdenCompraService</i> y <i>CompraClienteController</i> (403 si es de otro cliente).
RNF06	Todos los formularios están protegidos contra ataques CSRF y las acciones que modifican datos se envían por POST.	Protección CSRF de Spring Security activa; token incluido en los layouts y formularios.
RNF07	Al cerrar sesión se invalida la sesión y se elimina la cookie de sesión.	Configuración de logout en <i>SecurityConfig</i> .
RNF08	Solo se aceptan imágenes JPG, PNG o WEBP de hasta 2 MB.	Validación en <i>ImagenService</i> y límite de subida en <i>application.properties</i> .

2. Usabilidad e interfaz

ID	Requerimiento	Cómo se cumple
RNF09	El sitio institucional usa una maqueta HTML/CSS y la gestión administrativa un panel tipo dashboard, con un único template visual (basado en Bootstrap 4) y componentes comunes.	Layouts <i>publico.html</i> y <i>admin.html</i> , kit de fragments reutilizables y estilos de marca en <i>zero.css</i> .
RNF10	El sitio público es responsive: se puede usar desde celulares, tablets y computadoras.	Grilla responsive de Bootstrap 4 y meta viewport en los layouts.
RNF11	Toda la interfaz y los mensajes están en español. Los errores de validación se muestran con mensajes claros.	<i>ExceptionHandler</i> con mensajes en español mostrados como mensajes flash.

ID	Requerimiento	Cómo se cumple
RNF12	Los estados de pedidos y facturas se muestran siempre con el mismo color y etiqueta en todo el sistema.	Fragment <i>badge-estado</i> único.
RNF13	Ante un error, el usuario ve páginas de error propias del sistema (403, 404 y 500) en lugar de errores técnicos.	Plantillas <i>error/403</i> , <i>error/404</i> y <i>error/500</i> ; <i>GlobalExceptionHandler</i> .
RNF14	Los listados largos tienen filtros y paginación, y los reportes se pueden exportar a CSV e imprimir.	Fragments de filtros y paginación; <i>ExportadorCsv</i> y vista de impresión.

3. Rendimiento y disponibilidad

ID	Requerimiento	Cómo se cumple
RNF15	El envío de correos no demora la operación del usuario: se realiza en segundo plano y cada intento tiene un tiempo máximo de espera de 10 segundos.	<i>@Async</i> en <i>EmailService</i> (<i>AsyncConfig</i>) y timeouts SMTP de 10 000 ms.
RNF16	Las tareas automáticas se ejecutan sin intervención: el newsletter de ofertas se envía cada 10 días y diariamente se detectan los productos con precio sin actualizar hace más de 2 meses (horario de Argentina).	<i>NewsletterScheduler</i> (9:00) y <i>PrecioScheduler</i> (9:15), zona <i>America/Argentina/Buenos_Aires</i> .

4. Confiabilidad e integridad de datos

ID	Requerimiento	Cómo se cumple
RNF17	No se eliminan datos físicamente: las bajas son lógicas y los registros dados de baja dejan de mostrarse.	Atributo <i>eliminado</i> en <i>BaseEntity</i> ; métodos <i>eliminarX()</i> y <i>listarXActivo()</i> .
RNF18	La base de datos mantiene la integridad referencial y las operaciones de negocio se ejecutan de forma transaccional (se completan enteras o no se aplican).	SQLite con <i>foreign_keys=on</i> ; services con <i>@Transactional</i> .
RNF19	El stock nunca puede quedar negativo y una venta conserva el precio vigente al momento de la compra, aunque el precio cambie después.	Control en <i>StockService</i> ; el precio se guarda en cada <i>DetalleFactura</i> .
RNF20	Cada registro se identifica con un identificador único universal (UUID).	<i>@UuidGenerator</i> en <i>BaseEntity</i> .

5. Mantenibilidad y calidad

ID	Requerimiento	Cómo se cumple
RNF21	El software es web, con arquitectura Cliente-Servidor de al menos tres capas (presentación, negocio y persistencia) siguiendo el patrón MVC. Un controlador nunca accede directamente a la persistencia.	Capas controllers → services → repositories, vistas Thymeleaf.
RNF22	Se aplican patrones de software: Capas, MVC, DAO, DTO (reportes), Inyección de dependencias, Polimorfismo, Creador, Experto y Alta cohesión / Bajo acoplamiento.	Ver diagrama de paquetes y diagrama de clases de diseño.
RNF23	El código respeta convenciones comunes: nombres en español iguales al diagrama de clases, un service por entidad e inyección de dependencias por constructor.	Reglas del Plan de Trabajo y README del proyecto.

ID	Requerimiento	Cómo se cumple
RNF24	El sistema cuenta con pruebas automatizadas unitarias y de integración que se pueden ejecutar con un solo comando.	72 clases de prueba con 379 casos (JUnit 5, Mockito, Spring Test y GreenMail). Se ejecutan con <i>mvnw clean verify</i> .
RNF25	El código fuente se versiona en un repositorio Git y las tareas se organizan en un tablero Kanban.	Repositorio en GitHub; tablero en Trello.

6. Portabilidad, instalación y tecnología

ID	Requerimiento	Cómo se cumple
RNF26	Tecnologías: Java 21, Spring Boot 3.5, Thymeleaf, Spring Data JPA con Hibernate, SQLite, Spring Security, Spring Mail y Bootstrap 4 (incluido en el template).	Dependencias declaradas en pom.xml.
RNF27	La instalación solo requiere el JDK 21: no hace falta instalar Maven, un servidor de aplicaciones ni un motor de base de datos. Al iniciar por primera vez, la base se crea y se carga con datos iniciales.	Maven Wrapper, Tomcat embebido (puerto 8080), SQLite embebido en <i>data/zero.db</i> y DataSeeder.
RNF28	El servidor de correo (SMTP) y la URL pública del sitio se configuran sin modificar el código.	Pantalla de configuración de correo de la empresa; variable de entorno <i>APP_URL_BASE</i> .

7. Restricciones del negocio

ID	Requerimiento	Cómo se cumple
RNF29	Los pagos con Mercado Pago son simulados: no se realizan cobros reales ni se usan credenciales externas. El pago lo confirma un administrador desde el panel.	Forma de pago BILLETERA_VIRTUAL sin SDK externo; confirmación en el panel de pedidos.
RNF30	Las facturas se numeran en forma secuencial por tipo (cliente y proveedor), simulando la numeración validada por ARCA.	<i>siguienteNumero()</i> en VentaService y FacturaProveedorService.
RNF31	Para no aumentar los costos de desarrollo no se contratan diseñadores: el diseño se basa en un template HTML/CSS existente, adaptado por el equipo a la marca Zero.	Template en <i>static/vendor/template</i> .